

**LAPORAN PRAKTIKUM INDIVIDU
PERTEMUAN 4
TEKNOLOGI IOT II**

Disusun dalam rangka memenuhi salah satu syarat kelengkapan nilai Praktikum
Mata Kuliah Teknologi IoT II

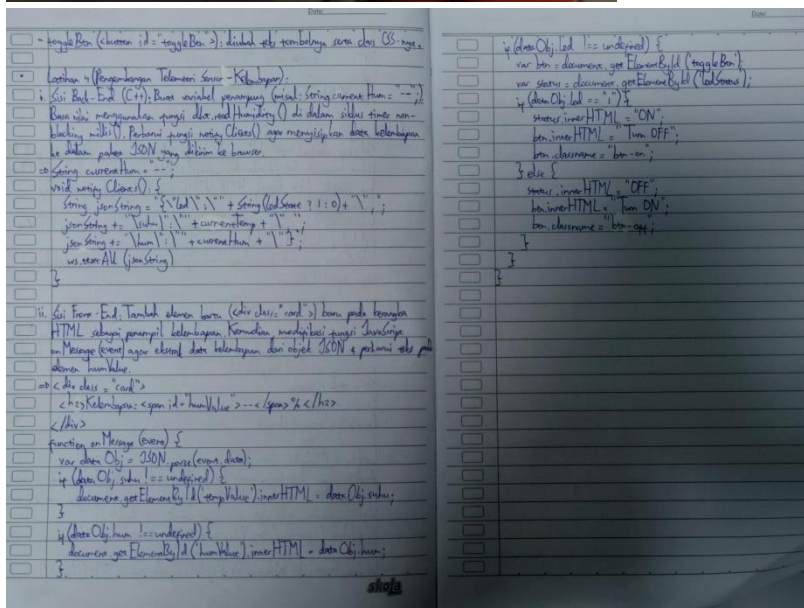
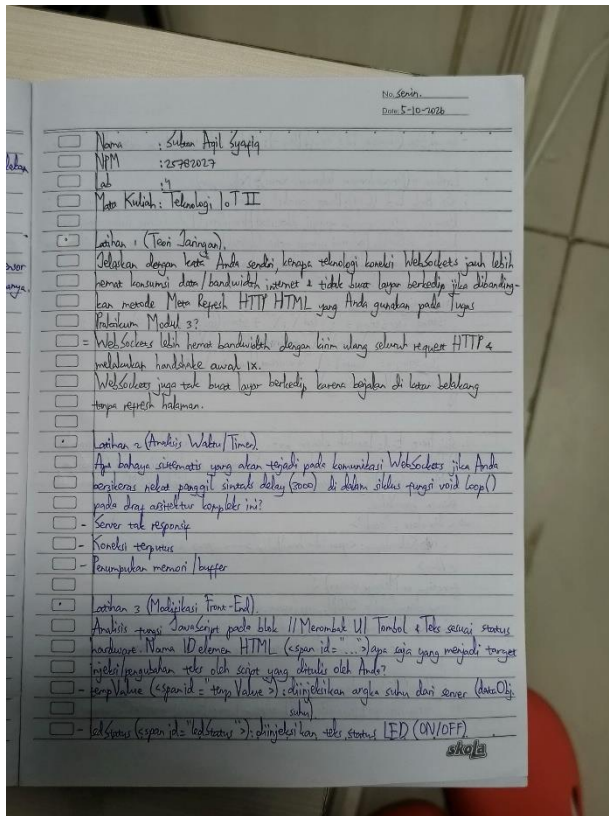


DISUSUN OLEH :

NAMA : SULTAN AQIL SYAFIQ.
NPM : 25782027
PRODI : TEKNOLOGI REKAYASA INTERNET
KELAS : 3A
DOSEN : Dr.Ir. Septafiansyah Dwi Putra, S.T., M.T.
Agus Ambarwari, S.Pd., M.Kom.

**KEMENTERIAN PENDIDIKAN TINGGI, SAINS, DAN
TEKNOLOGI
POLITEKNIK NEGERI LAMPUNG
TAHUN AJARAN 2026 / 2027**

❖ LOGBOOK:



❖ Praktikum 1: Real-Time Dashboard & Sinkronisasi Tombol Fisik).

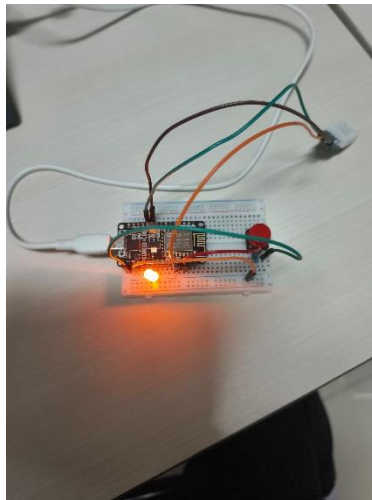
Langkah Kerja Rangkaian:

1. Hubungkan kaki **Anoda** (kaki panjang) LED ke pin **D6 (GPIO 12)**.
 Hubungkan kaki **Katoda** (kaki pendek) LED ke salah satu kaki **Resistor 220 Ohm**, lalu hubungkan kaki resistor lainnya ke pin **GND NodeMCU** (sama seperti rangkaian LED pada Modul 3).

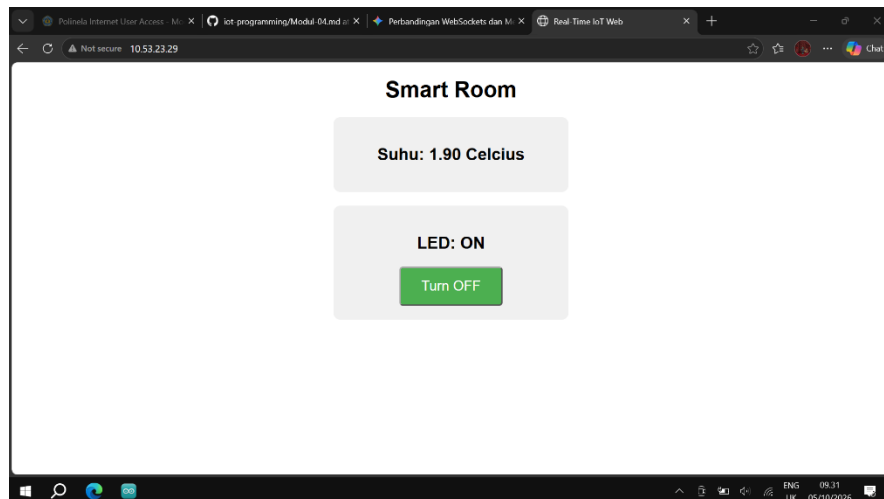
2. Hubungkan pin **DATA / OUT** DHT ke pin **D4 (GPIO 2)**. (VCC DHT ke 3V3, GND ke GND).
3. Pasang Push Button pada breadboard, melintasi parit tengah. Hubungkan salah satu kakinya ke jalur 3V3 (3.3V). Hubungkan kaki seberang tombol ke pin D2 (GPIO 4) NodeMCU. Pada titik kaki yang terhubung ke pin D2 tersebut, pasang salah satu kaki Resistor 10k Ohm, lalu hubungkan ujung kaki resistor lainnya ke jalur GND (konfigurasi resistor pull-down eksternal seperti pada Modul 1).

- **SCREENSHOT PRAKTIKUM 1:**

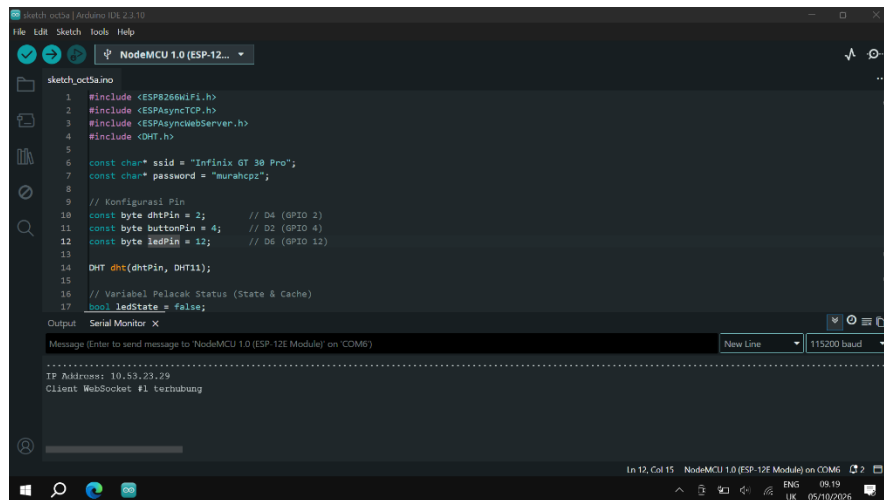
- *Foto rangkaian:*



- *Dashboard web:*



- *Serial Monitor:*



❖ **TUGAS MANDIRI:**

- Sinyal Digital (*ON/OFF*) sangat berguna, namun bagaimana jika Anda ditugaskan merancang antarmuka penyesuaian intensitas redup-terang lampu (*Dimmer*) dari *smartphone*?
- **Skenario Proyek (*Smart Lighting*):** Anda akan mengeksplorasi penggunaan modulasi sinyal listrik `analogWrite()` (atau dikenal dengan fitur PWM) dan memadukannya dengan teknologi masukan elemen *Slider / Range* dari HTML5.
 - **Tujuan:** Menguasai transmisi paket String WebSocket berkelanjutan (continuous packet) dari manipulasi antarmuka layar slider untuk merombak parameter persentase daya komponen aktuatur.
 - **Tingkat Kesulitan:** Lanjut
 - **Instruksi Perangkaan:**
 - i). Tambahkan sebuah komponen LED yang dilengkapi Resistor 220 Ohm (sebagai pengaman arus).
 - ii). Hubungkan kaki Anoda (positif) sistem LED tersebut ke pin D1 (GPIO 5).
 - **Instruksi Analisis Arsitektur:**
 - i). Modifikasi desain kerangka web HTML Anda. Sisipkan elemen input Slider HTML. (Hint sintaksis: `<input type="range" min="0" max="1023" id="pwmSlider" onchange="sendPWM(this.value)">`)

- ii). Buat sebuah fungsi JavaScript bernama `sendPWM(value)` yang akan mengirimkan paket *string* teks berformat khusus (misalnya "pwm,512") ke NodeMCU melalui *websocket*.
- iii). Pada ruang fungsi C++ `handleWebSocketMessage()` di sisi server (NodeMCU), pecah (*parsing*) atau periksa string tersebut menggunakan fitur deteksi *substring* atau bahasa pemrosesan C++ lainnya.
- iv). Jika Server NodeMCU sukses mendeteksi dan mengambil angka murninya, panggil fungsi aktuator `analogWrite(pinLED, nilaiPWM)` untuk secara elegan meredupkan atau menerangkan LED LED fisik secara seketika sesuai gesekan layar Anda!
- **Kriteria Keberhasilan:**
 - Di layar gawai Anda, hadir tuas gulir horizontal (*Slider*).
 - LED di dunia nyata dapat diredupkan dan diterangkan secara bertahap saat Anda menggeser tuas slider, bebas dari lag (real-time response).
 - Kendali LED dan pembacaan Sensor DHT bawaan Praktikum 1 dipastikan masih bekerja berdampingan dengan sempurna.

❖ KODE TUGAS MANDIRI:

```
#include <ESP8266WiFi.h>
#include <ESPAsyncTCP.h>
#include <ESPAsyncWebServer.h>
#include <DHT.h>

const char* ssid = "Infinix GT 30 Pro";
const char* password = "murahcpz";

// Konfigurasi Pin
const byte dhtPin = 2;           // D4 (GPIO 2)
const byte buttonPin = 4;        // D2 (GPIO 4)
const byte ledPin = 12;          // D6 (GPIO 12)
const byte pwmLedPin = 5;        // D1 (GPIO 5) -> Untuk PWM Dimmer

DHT dht(dhtPin, DHT11);

// Variabel Pelacak Status (State & Cache)
bool ledState = false;
int pwmValue = 0;                // Menyimpan nilai PWM (0 - 1023)
String currentTemp = "--";
```

```

String currentHum = "--";

int buttonState;
int lastButtonState = LOW;
unsigned long lastDebounceTime = 0;
const unsigned long debounceDelay = 50;
unsigned long lastTime = 0;

// Inisialisasi Async Web Server & WebSocket
AsyncWebServer server(80);
AsyncWebSocket ws("/ws");

// ----- HTML, CSS, & JAVASCRIPT (FRONT-END) -----
const char index_html[] PROGMEM = R"rawliteral(
<!DOCTYPE html>
<html>
<head>
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Real-Time IoT Web & PWM Dimmer</title>
  <style>
    body { font-family: Arial; text-align: center; background-color: #f4f4f9; margin: 0;
padding: 20px; }
    .card { background: #ffffff; margin: 15px auto; padding: 20px; max-width: 350px;
border-radius: 12px; box-shadow: 0 4px 8px rgba(0,0,0,0.1); }
    button { padding: 12px 25px; font-size: 18px; border-radius: 6px; border: none; cursor:
pointer; color: white; margin-top: 10px; }
    .btn-on { background-color: #4CAF50; }
    .btn-off { background-color: #f44336; }
    .slider { width: 80%; margin: 15px 0; }
  </style>
</head>
<body>
  <h1>Smart Room & Lighting</h1>

  <div class="card">
    <h2>Suhu: <span id="tempValue">--</span> &deg;C</h2>
    <h2>Kelembapan: <span id="humValue">--</span> %</h2>
  </div>

  <div class="card">
    <h2>LED Sakelar: <span id="ledStatus">OFF</span></h2>
    <button id="toggleBtn" class="btn-off" onclick="toggleLed()">Turn ON</button>
  </div>

  <!-- Element Slider PWM Dimmer -->
  <div class="card">
    <h2>LED Dimmer (PWM)</h2>
    <p>Intensitas: <span id="pwmValText">0</span> / 1023</p>

```

```

    <input type="range" min="0" max="1023" value="0" class="slider" id="pwmSlider"
oninput="sendPWM(this.value)">
</div>

<script>
    var gateway = `ws://${window.location.hostname}/ws`;
    var websocket;

    window.addEventListener('load', onLoad);
    function onLoad(event) { initWebSocket(); }

    function initWebSocket() {
        websocket = new WebSocket(gateway);
        websocket.onopen = onOpen;
        websocket.onclose = onClose;
        websocket.onmessage = onMessage;
    }

    function onOpen(event) { console.log('WebSocket Terkoneksi'); }
    function onClose(event) { setTimeout(initWebSocket, 2000); }

    function toggleLed(){
        websocket.send('toggle');
    }

    // Mengirimkan data slider dengan format "pwm,nilai" ke NodeMCU
    function sendPWM(value){
        document.getElementById('pwmValText').innerHTML = value;
        websocket.send('pwm,' + value);
    }

    function onMessage(event) {
        var dataObj = JSON.parse(event.data);

        if(dataObj.suhu !== undefined) {
            document.getElementById('tempValue').innerHTML = dataObj.suhu;
        }

        if(dataObj.hum !== undefined) {
            document.getElementById('humValue').innerHTML = dataObj.hum;
        }

        if(dataObj.led !== undefined) {
            var btn = document.getElementById('toggleBtn');
            var status = document.getElementById('ledStatus');
            if(dataObj.led == "1"){
                status.innerHTML = "ON";
                btn.innerHTML = "Turn OFF";
            }
        }
    }

```

```

        btn.className = "btn-on";
    } else {
        status.innerHTML = "OFF";
        btn.innerHTML = "Turn ON";
        btn.className = "btn-off";
    }
}

if(dataObj.pwm !== undefined) {
    document.getElementById('pwmSlider').value = dataObj.pwm;
    document.getElementById('pwmValText').innerHTML = dataObj.pwm;
}
}
</script>
</body>
</html>
)rawliteral";

// ----- BACK-END & WEBSOCKET LOGIC -----

void notifyClients() {
    String jsonString = "{\"led\":\"" + String(ledState ? 1 : 0) + "\", ";
    jsonString += "\"suhu\":\"" + currentTemp + "\", ";
    jsonString += "\"hum\":\"" + currentHum + "\", ";
    jsonString += "\"pwm\":\"" + String(pwmValue) + "\"}";
    ws.textAll(jsonString);
}

void handleWebSocketMessage(void *arg, uint8_t *data, size_t len) {
    AwsFrameInfo *info = (AwsFrameInfo*)arg;
    if (info->final && info->index == 0 && info->len == len && info->opcode == WS_TEXT) {
        data[len] = 0;
        String message = String((char*)data);

        // 1. Opsi Toggle LED Digital
        if (message == "toggle") {
            ledState = !ledState;
            notifyClients();
        }

        // 2. Opsi Parsing String Slider PWM (Format: "pwm,nilai")
        else if (message.startsWith("pwm,")) {
            String valueStr = message.substring(4); // Mengambil karakter setelah "pwm,"
            pwmValue = valueStr.toInt();             // Konversi teks angka ke integer

            analogWrite(pwmLedPin, pwmValue);        // Mengubah redundansi intensitas daya LED
(PWM)
            notifyClients();                          // Sinkronkan ke seluruh browser lain

```

```

    }
}

void onEvent(AsyncWebSocket *server, AsyncWebSocketClient *client, AwsEventType type,
             void *arg, uint8_t *data, size_t len) {
    switch (type) {
        case WS_EVT_CONNECT:
            Serial.printf("Client WebSocket #%u terhubung\n", client->id());
            notifyClients();
            break;
        case WS_EVT_DISCONNECT:
            Serial.printf("Client WebSocket #%u terputus\n", client->id());
            break;
        case WS_EVT_DATA:
            handleWebSocketMessage(arg, data, len);
            break;
    }
}

void setup() {
    Serial.begin(115200);

    pinMode(buttonPin, INPUT);
    pinMode(ledPin, OUTPUT);
    pinMode(pwmLedPin, OUTPUT);

    digitalWrite(ledPin, LOW);
    analogWrite(pwmLedPin, pwmValue); // Inisialisasi awal PWM = 0

    dht.begin();

    WiFi.mode(WIFI_STA);
    WiFi.begin(ssid, password);
    while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
    Serial.println("\nIP Address: " + WiFi.localIP().toString());

    server.on("/", HTTP_GET, [](AsyncWebServerRequest *request){
        request->send_P(200, "text/html", index_html);
    });

    ws.onEvent(onEvent);
    server.addHandler(&ws);
    server.begin();
}

void loop() {
    ws.cleanupClients();
}

```

```

// 1. Eksekusi LED Digital
digitalWrite(ledPin, ledState ? HIGH : LOW);

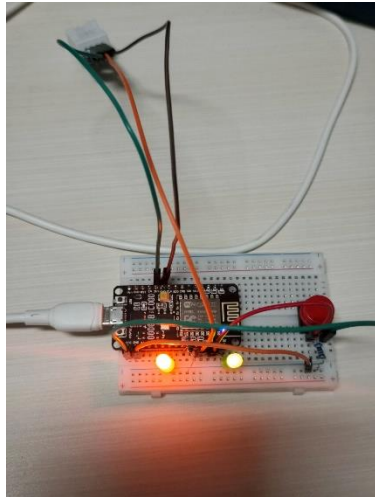
// 2. Debounce Tombol Fisik
int reading = digitalRead(buttonPin);
if (reading != lastButtonState) {
    lastDebounceTime = millis();
}
if ((millis() - lastDebounceTime) > debounceDelay) {
    if (reading != buttonState) {
        buttonState = reading;
        if (buttonState == HIGH) {
            ledState = !ledState;
            notifyClients();
        }
    }
}
lastButtonState = reading;

// 3. Pembacaan Sensor Non-Blocking (Suhu & Kelembapan)
if ((millis() - lastTime) > 3000) {
    float t = dht.readTemperature();
    float h = dht.readHumidity();
    if(!isnan(t) && !isnan(h)) {
        currentTemp = String(t);
        currentHum = String(h);
        notifyClients();
    }
    lastTime = millis();
}
}

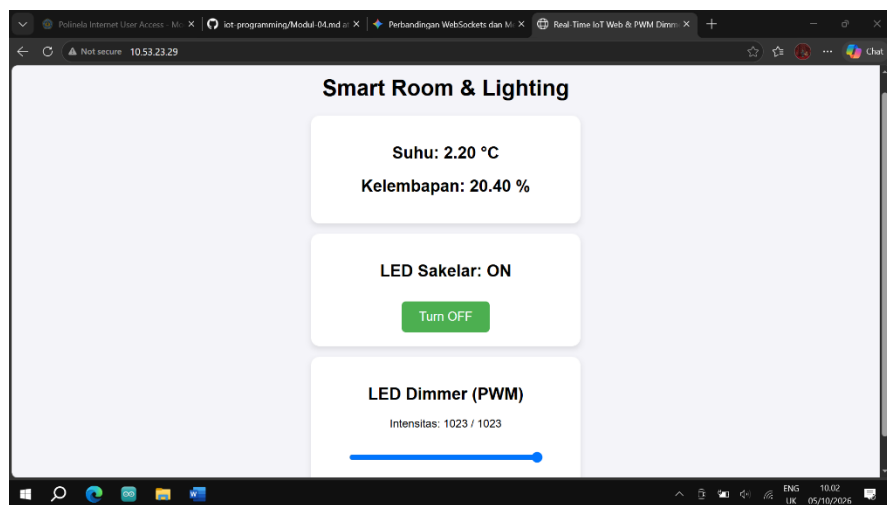
```

❖ **SCREENSHOT TUGAS MANDIRI:**

- *Foto rangkaian:*



- *Dashboard web:*



- *Serial Monitor:*

